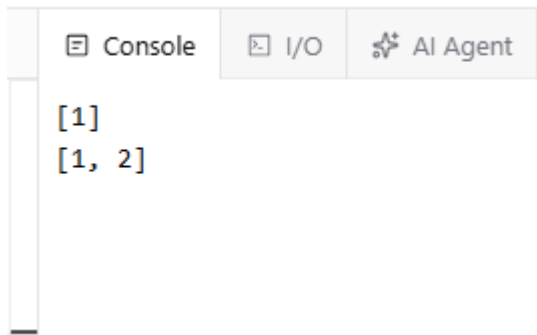


## 1) Python

a)



```
Console I/O AI Agent  
[1]  
[1, 2]
```

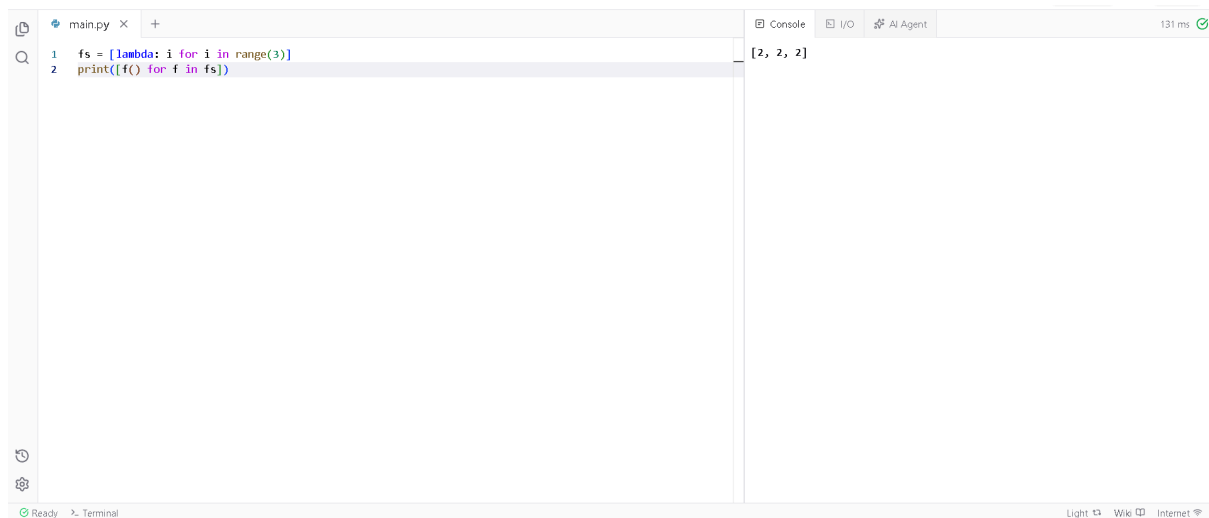
b) Usa o conceito de passagem por atribuição e parâmetros padrão

## 2) Java

a) A saída é "0 5"

b) Em Java, como Arrays são objetos, é possível alterar os valores dentro dele por funções (referência). Já a variável n não é trocada, pois a passagem é apenas por valor

## 3) Python -



```
main.py x + Console I/O AI Agent 131 ms  
1 fs = [lambda: i for i in range(3)]  
2 print([f() for f in fs])  
[2, 2, 2]
```

b) Conceito de fechamentos, As lambdas capturam a VARIÁVEL i, que no fim vale 2 (late binding). Correção: lambda i=i, o mesmo problema do var em JS.

## 4)C -

The screenshot shows the OneCompiler C IDE. The editor contains the following C code:

```
1 #include <stdio.h>
2 int contador(void) {
3     static int n = 0;
4     return ++n;
5 }
6 int main()
7 {
8     contador(); contador();
9     printf("%d\n", contador());
10 }
11
12
```

The console on the right shows the output: 3. The status bar at the bottom indicates 'Ready' and 'Terminal'.

b) Variáveis estáticas,

## 5) Rust

The screenshot shows the OneCompiler Rust IDE. The editor contains the following Rust code:

```
1 fn main() {
2     fn dobra(v: Vec<i32>) -> Vec<i32> {
3         v.iter().map(|x| x * 2).collect()
4     }
5
6     let v = vec![1, 2, 3];
7     let d = dobra(v);
8     println!("{}", v, d);
9 }
10
```

The console on the right shows the following error:

```
error[E0382]: borrow of moved value: 'v'
--> HelloWorld.rs:7:23
|
5 | let v = vec![1, 2, 3];
|   - move occurs because 'v' has type 'Vec<i32>', which does not implement the 'Copy' trait
6 | let d = dobra(v);
|           - value moved here
7 | println!("{}", v, d);
|           ^ value borrowed here after move

note: consider changing this parameter type in function 'dobra' to borrow instead if owning the value isn't necessary
--> HelloWorld.rs:2:17
|
2 | fn dobra(v: Vec<i32>) -> Vec<i32> {
|   ----- this parameter takes ownership of the value
|   |
|   | in this function
|   | help: consider cloning the value if the performance cost is acceptable
6 | let d = dobra(v.clone());
|           ++++++++

error: aborting due to 1 previous error

For more information about this error, try 'rustc --explain E0382'.
Debug this error
```

a) Saída: Não há saída, pois o código não compila devido a um erro estrutural.

b) Conceito de passagem de variáveis: o erro é causado porque a variável V é transportada para o subprograma e não pode mais ser acessada de fora dele.

## 6) Python -

a)

Console

I/O

AI Agent

Exit 1

```
Traceback (most recent call last):
  File "/box/main.py", line 7, in <module>
    print(adiciona(5))
    ^^^^^^^^^^^
  File "/box/main.py", line 4, in adiciona
    total = total + x
    ^^^^^
UnboundLocalError: cannot access local variable 'total' where it
is not associated with a value
```

 [Debug this error](#)

b) Conceito: ambiente de referenciamento local